

Cookie, CSRF

Wykłady 8

Bartosz Lauks

30 kwietnia 2026



① Cookie

② CSRF

① Cookie

② CSRF

Co to są pliki cookie?

- Pliki cookie (ciasteczka) to niewielkie porcje danych, które serwer przesyła do przeglądarki internetowej użytkownika i które są przez nią przechowywane lokalnie na urządzeniu użytkownika.
- Za każdym razem, gdy użytkownik ponownie odwiedza daną stronę, jego przeglądarka automatycznie przesyła zapisane cookie z powrotem do serwera.

Cechy charakterystyczne cookie

- Są przechowywane po stronie klienta (w przeglądarce użytkownika).
- Mogą przechowywać niewielkie ilości danych (ok. 4 KB).
- Są powiązane z domeną i ścieżką. Przeglądarka wysyła je tylko do odpowiednich serwerów.
- Mogą mieć określony czas życia (cookies sesyjne lub trwałe).
- Mogą być odczytywane przez serwer oraz (opcjonalnie) przez JavaScript.
- Są wykorzystywane m.in. do zarządzania sesją, personalizacji oraz śledzenia użytkownika.

Typowe zastosowania plików cookie w aplikacjach webowych

- **Identyfikacja użytkownika** (np. sesje logowania).
- **Zapamiętywanie ustawień** (np. język strony, motyw graficzny).
- **Koszyki zakupowe** w e-commerce.
- **Śledzenie aktywności użytkownika** (np. Google Analytics).
- **Reklama spersonalizowana** (targetowanie reklam na podstawie zachowań użytkownika).

Cookie

Jak działają cookie w praktyce?

- Podczas pierwszego kontaktu z serwerem:
 - Serwer wysyła nagłówek **Set-Cookie**, aby utworzyć cookie w przeglądarce.
 - Przeglądarka zapisuje cookie i przesyła je (nagłówek **Cookie**) z powrotem przy każdej kolejnej prośbie (żądaniu HTTP) do tej samej domeny.

Listing 1: Nagłówek response serwera. Nakazujący przeglądarce utworzenie ciastka (cookie)

```
1 Set-Cookie: username=JanKowalski; expires=Tue, 01-May-2025 12:00:00 GMT  
   ; path=/; secure; HttpOnly
```

Obsługa plików cookie w języku PHP

- PHP oferuje wbudowane funkcje do obsługi cookie zarówno do ich tworzenia, jak i odczytu.

`setcookie(name, value, expire, path, domain, secure, httponly);`

- Parametry:
 - **name** – nazwa cookie.
 - **value** – wartość cookie (tekst).
 - **expire** – czas wygaśnięcia (znacznik czasu UNIX).
 - **path** – ścieżka w obrębie domeny, dla której cookie jest dostępne.
 - **domain** – opcjonalnie określona domena.
 - **secure** – czy cookie ma być przesyłane tylko przez HTTPS.
 - **httponly** – czy cookie ma być dostępne tylko z poziomu HTTP (a nie JavaScriptu).

Cookie

Praktyczny przykład tworzenia cookie przy pomocy PHP

- Przy pierwszym wykonaniu poniższego skryptu w **DevTools** sprawdź nagłówek **Set-Cookie** w **response**.
- Później wykonaj kolejny **request** pod dowolny URL dla domeny **localhost** i sprawdź nagłówek **Cookie**.

Listing 2: Tworzenie cookie w PHP

```
1 setcookie("username", "", time() - 3600, "/");
```

Przykład: localhost/lecture8/cookie/create.php

Odczytywanie cookie w PHP

- Wszystkie cookie są dostępne w **superglobalnej** tablicy **`$_COOKIE`**.
- Dzięki temu w łatwy sposób możemy operować na danych ciastek po stronie serwera przy użyciu PHP.
- **Uwaga!** Zmiana danych cookie w superglobalnej nie spowoduje przesłanie nowej wersji do przeglądarki klienta, aby to zrobić należy użyć **`setcookie()`**.

Przykład: localhost/lecture8/cookie/check.php

Cookie

Usuwanie cookie

- Aby usunąć cookie, należy ustawić jego czas wygaśnięcia w przeszłości:

Listing 3: Usuwanie cookie w PHP

```
1 setcookie("username", "", time() - 3600, "/");
```

Przykład: localhost/lecture8/cookie/remove.php

Cookie a sesje w PHP oraz bezpieczeństwo.

- Warto zaznaczyć, że choć cookies i sesje to różne mechanizmy, PHP domyślnie wykorzystuje cookies do obsługi sesji.
- Po stronie klienta tworzony jest cookie zawierający identyfikator sesji (**PHPSESSID**), który służy do odczytu danych sesyjnych przechowywanych po stronie serwera.
- Dlatego często stają się one celem ataków, które mogą prowadzić do przejęcia danych w nich zawartych.
- Z tego powodu ciasteczka posiadają kilka mechanizmów i właściwości, które pozwalają w znacznym stopniu je zabezpieczyć.

Cookie bezpieczeństwo.

- Pierwszym, domyślnym zabezpieczeniem jest standard bezpieczeństwa przeglądarek, które przesyłają tylko te ciasteczka, które zostały utworzone przez daną domenę.
- Dzięki temu mamy pewność, że przeglądarka nie prześle ciasteczek sesyjnych do innej strony podstawionej przez atakującego.

Cookie

Cookie bezpieczeństwo. (Secure)

- Jedną z ważnych właściwości jest **Secure**.
- Ustawienie jej na **true** zapewnia, że przeglądarka prześle cookie tylko wtedy, gdy korzystamy z zaszyfrowanego protokołu HTTPS.
- Zabezpiecza to dane przed ich odczytaniem przez narzędzia służące do podsłuchiwania sesji.

Przykład: localhost/lecture8/cookie/security/secure

Cookie bezpieczeństwo. (HttpOnly)

- Kolejną istotną właściwością jest **HttpOnly**.
- Na laboratoriach poznaliśmy już możliwość odczytu i modyfikacji ciasteczek w JavaScript za pomocą obiektu **document.cookie**.
- Ustawienie tej właściwości na **true** powoduje, że ciasteczko nie jest dostępne z poziomu JavaScript.
- Dzięki temu mechanizm ten chroni przed atakami polegającymi na próbie wykradania lub manipulowania danymi przechowywanymi w ciasteczkach przy użyciu JavaScript.

Przykład: <localhost/lecture8/cookie/security/httpOnly>

Cookie

Cookie bezpieczeństwo. (SameSite)

- Następną ważną właściwością jest **SameSite**.
- Określa ona, czy ciasteczko może być wysyłane wraz z żądaniami między różnymi stronami.
- Może przyjmować wartości:
 - **SameSite=Strict** - Cookie są wysyłane tylko wtedy, gdy użytkownik nawiguje w obrębie tej samej domeny
 - **SameSite=Lax** - Cookie są wysyłane w przypadku tzw. nawigacji nawigacyjnej, czyli: kliknięcia linku do Twojej strony, odświeżenia strony, wpisania adresu w przeglądarce. (**HTTP GET**)
 - **SameSite=None** - Cookie są wysyłane zawsze, nawet w przypadku żądań z obcych domen (cross-site), osadzonych **iframe**, **API (fetch())** i **AJAX** z innych stron.

Przykład: localhost/lecture8/cookie/security/sameSite

Zakres dostępu cookie: Path

- Jedną z istotnych właściwości określających dostęp aplikacji do ciasteczek jest **Path**.
- Określa ona, dla jakiej ścieżki (URL) w obrębie serwera cookie będzie przesyłane.
- Przeglądarka dołącza ciasteczko tylko do żądań kierowanych do zasobów znajdujących się w określonej ścieżce.
- Przykładowo ustawienie `/admin` sprawi, że cookie będzie przesyłane tylko dla zasobów w katalogu `/admin`.

Przykład: `localhost/lecture8/cookie/security/path`

Zakres dostępu cookie: Domain

- Ostatnią właściwością jest **Domain**, która określa, dla jakiej domeny cookie jest dostępne.
- Może obejmować zarówno konkretną domenę, jak i jej subdomeny (np. [.example.com](http://example.com)).
- Uwaga! Nie oznacza to, że właściwość ta pozwala na ustawienie ciasteczka dla dowolnej domeny.
- Mechanizm ten umożliwia jedynie współdzielenie cookies w obrębie domeny i jej subdomen.

Przykład: localhost/lecture8/cookie/security/domain

Ograniczenia i alternatywy

- **Widoczność dla klienta** – dane w cookies nie są szyfrowane, można je łatwo podejrzeć.
- **Ograniczenia przestrzeni** – cookies mogą mieć maksymalnie około 4 KB oraz ograniczoną liczbę dla jednej domeny (zwykle 20–50, w zależności od przeglądarki).
- Cookies nie nadają się do przechowywania dużych struktur danych. W tym celu lepsze są **localStorage** oraz **sessionStorage**.
- Należy stosować je wyłącznie jako niewielkie pojemniki na dane, ponieważ nie posiadają własnych mechanizmów bezpieczeństwa.

① Cookie

② CSRF

CSRF

Czym jest ?

- **Cross-Site Request Forgery (CSRF)** to jedno z najgroźniejszych i jednocześnie bardzo podstępnych zagrożeń bezpieczeństwa aplikacji webowych.
- W skrócie, CSRF polega na „wymuszeniu” przez atakującego wykonania nieautoryzowanego działania w imieniu zalogowanego użytkownika, bez jego wiedzy i zgody.

Przykład ataku

- Wyobraźmy sobie, że jesteśmy zalogowany na swoje konto bankowe i równocześnie odwiedzasz inną stronę.
- Ta strona wykorzystując to, że w domenie banku mamy utworzoną sesję, potajemnie wysyła przelew z naszego konta. To właśnie CSRF.

Skutki ataku CSRF

- Zmiana hasła użytkownika.
- Wysyłanie wiadomości (np. w serwisach społecznościowych).
- Wykonywanie przelewów bankowych.
- Modyfikacja danych w systemie.
- Zmiana adresu e-mail powiązanego z kontem.

Wszystko to bez wiedzy użytkownika, ale za jego autoryzacją (**ciasteczko sesyjne**).

CSRF

Wektory ataku

- Złośliwe formularze HTML z **POST**:

Listing 4: Złośliwe formularze

```
1 <form action="https://mojbank.pl/przelew" method="POST">
2   <input type="hidden" name="konto" value="12345678">
3   <input type="hidden" name="kwota" value="10000">
4   <input type="submit" value="Kliknij tu!">
5 </form>
6 <script>document.forms[0].submit();</script>
```

Wektory ataku cd.

- Złośliwe odwołania do obrazków / iframe'ów / linków (**GET**).

Listing 5: Złośliwe odwołania do obrazków

```
1 
2
3 <iframe src="https://mojbank.pl/przelew?konto=12345678&kwota=10000"
  title="Iframe Example"></iframe>
```


CSRF

Mechanizm działania

- Warunki, które muszą być spełnione:
 - Ofiara jest zalogowana do aplikacji (np. banku, portalu społecznościowego).
 - Ofiara odwiedza złośliwą stronę (przygotowaną przez atakującego).
 - Przeglądarka ofiary automatycznie dołącza ciasteczko sesyjne do każdego żądania HTTP do aplikacji, w której użytkownik jest zalogowany.
 - Atakujący wykorzystuje zaufanie aplikacji do użytkownika.
 - Aplikacja nie rozpoznaje, że żądanie zostało wykonane z obcego źródła.

Przykład: localhost/lecture8/csrf

CSRF

Przykład. Co tu się dzieje?

- Użytkownik jest zalogowany do [bank.php](#) i ma ciasteczko sesyjne.
- Użytkownik odwiedza [atak.html](#).
- Formularz POST z [atak.html](#) zostaje wysłany automatycznie do [bank.php](#).
- [bank.php](#) odczytuje dane z `$_POST` i wykonuje przelew myśląc, że użytkownik sam zainicjował operację.

Zabezpieczenia przed CSRF

- **SameSite cookies** - Ustawienie flagi SameSite na ciasteczkach.
- Powoduje, że przeglądarka nie dołączy ciasteczka do żądań pochodzących z innej domeny.
- SameSite można wymusić dla wszystkich ciastek w pliku konfiguracyjnym [php.ini](#)

Listing 6: SameSite cookies

```
1 Set-Cookie: sessionid=abc123; SameSite=Strict
```

Zabezpieczenia przed CSRF z użyciem nagłówków **Referer** i **Origin**

- Do zabezpieczenia aplikacji przed atakiem **CSRF** często używa się nagłówków **Referer** oraz **Origin**.
- **Referer** - zawiera pełny adres **URL** strony źródłowej, czyli: **protokół (https), domenę oraz ścieżkę**.
Referer: <https://example.com/form.html>
- Jest przekazywany w żądaniu przez przeglądarkę, aby poinformować odbiorcę, pod jakim adresem (**URL**) znajdował się klient przed wysłaniem żądania.
- Niestety **Referer** nie jest w 100% niezawodny, ponieważ może zostać usunięty przez przeglądarkę lub ukryty przez politykę prywatności.

Zabezpieczenia przed CSRF z użyciem nagłówków **Referer** i **Origin** cd.

- Lepszym wyborem będzie nagłówek **Origin**, który składa się z: protokołu, domeny oraz portu.
Origin: <https://example.com>
- Jest wysyłany przez przeglądarkę przy **żądaniach POST** oraz zapytaniach **AJAX (fetch, XMLHttpRequest)**.
- **Origin** jest znacznie bardziej „stabilny” niż **Referer** i zazwyczaj nie jest modyfikowany ani usuwany przez mechanizmy prywatności w taki sam sposób.

Zabezpieczenia przed CSRF z użyciem nagłówków **Referer** i **Origin**

- Zabezpieczenie CSRF z użyciem nagłówków **Referer** i **Origin** jest prostym mechanizmem ochrony polegającym na sprawdzaniu źródła żądania.
- Przeglądarka dodaje nagłówki **Referer** lub **Origin** do żądań wychodzących (np. formularzy **POST**).
- Aplikacja może sprawdzić, czy te nagłówki wskazują na zaufaną domenę (np. **https://mojbank.pl**). Jeśli żądanie pochodzi z innej domeny, zostaje odrzucone.

Przykład – zabezpieczenie bank.php przy użyciu Origin:

Przykład: <localhost/lecture8/csrf/bankOrigin.php>

CSRF

Zabezpieczenia przed CSRF cd.

- Ostatnim omawianym sposobem na obronę przed atakami przed CSRF będzie **Token CSRF (anti-CSRF tokens)**
- Najpopularniejsze i skuteczne zabezpieczenie.
- Serwer generuje unikalny token, który umieszcza w formularzu HTML.
- Przy wysyłaniu żądania klient musi ten token odesłać.
- W ten sposób mamy pewność, że formularz przesłany w request na pewno pochodzi z naszego zaufanego serwera.

Listing 7: Token anty-csrf w formularzu

```
1 <input type="hidden" name="csrf_token" value="abc123xyz" />
```

Nowa funkcjonalność podatna na CSRF

- Wykorzystamy system z poprzedniego wykładu, aby zaimplementować **anti-CSRF tokens**.
- Stożymy nową podstronę, które umożliwi użytkownikowi zmianę hasła.
- Następnie zakatujemy serwis w ten sposób, aby bez wiedzy użytkownika podmienić jego hasło.
- Kolejno zabezpieczymy naszą nową funkcjonalności przed atakami tego typu i jeszcze raz przetestujemy działanie.

Przykład: localhost/lecture8/changePass.php

Atak CSRF na nową funkcjonalność

- Teraz wykonujemy atak CSRF.
- Zwabiamy użytkownika pod adres URL, gdzie znajduje się automatycznie wysyłający się formularz.
- Dzięki temu, że obiekt naszego ataku jest zalogowany wystarczy podstawić nowe dane pod pole input i wysłać żądanie zmiany hasła.
- Teraz przy próbie logowania starym hasłem serwis poinformuje nas że dane autoryzacji są nieprawidłowe.

Przykład: localhost/lecture8/changePassAttack.php

Implementacja anti-CSRF tokens

- Nowa wersja formularza rejestracji dodaje do formularza ukryty token.
- Token jest generowany po stronie serwera podczas tworzenia formularza i następnie do niego dodawany jako pole ukryte.
- Wykorzystuje się w tym celu atrybut `type='hidden'`, a jego wartość jest zapisywana w zmiennej sesyjnej `$_SESSION['csrf_token']`.
- Każde żądanie zmiany hasła zawiera teraz również ten token.
- Podczas przetwarzania danych sprawdzane jest, czy przesłany token jest identyczny z wersją przechowywaną na serwerze. W przypadku niezgodności zmiana zostaje odrzucona.

Przykład: localhost/lecture8/changePass.php

Wskazówki produkcyjne

- Do porównywania tokenów używaj funkcji `hash_equals()`, która chroni przed atakami typu **timing attack**. Nie używaj operatorów logicznych (`===`)
- Tokeny CSRF powinny być regenerowane cyklicznie, np. po zalogowaniu użytkownika lub odświeżeniu sesji.
- Dobrą praktyką jest stosowanie osobnego tokenu dla każdego formularza.
- Token CSRF powinien być powiązany z sesją użytkownika i walidowany po stronie serwera dla każdego żądania modyfikującego dane.
- W aplikacjach AJAX warto przysyłać token również w nagłówkach, np. **X-CSRF-TOKEN**.

Dziękuję za uwagę !

Bartosz Lauks